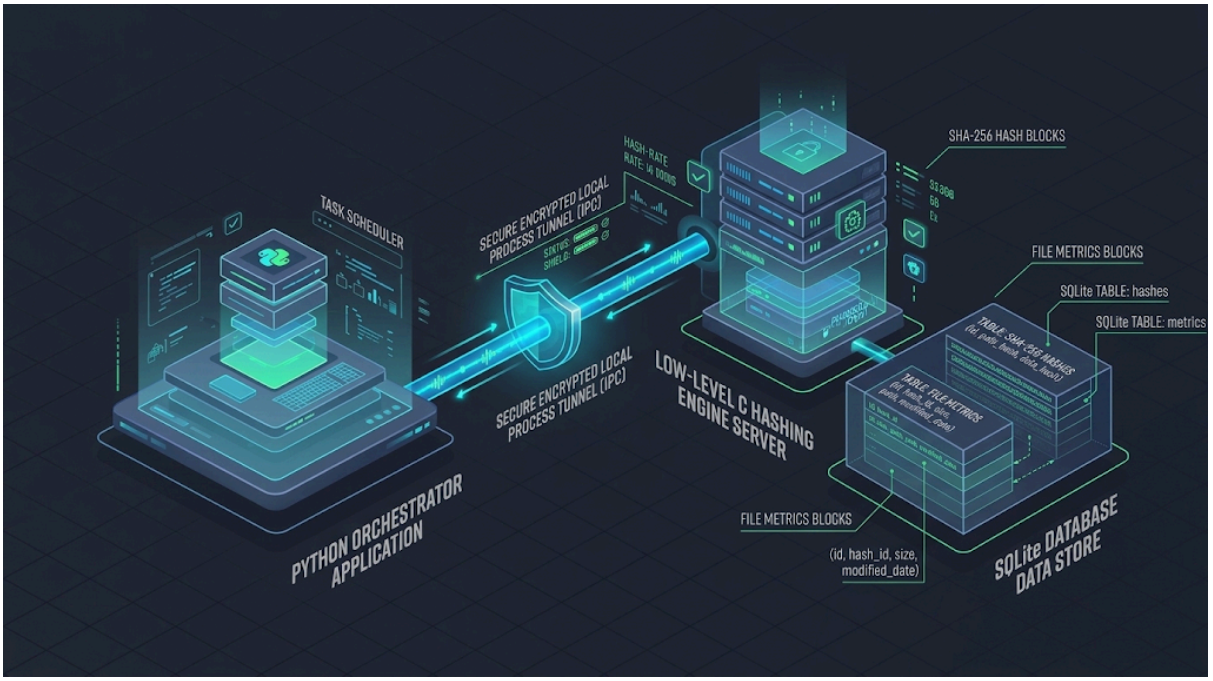
Universidad Nacional de Las Ciencias
Ingeniería en Ciberseguridad
Programación
Alumnos:
Nelson Sánchez, Cédula: 19.264.908
Andrea Pernil, Cédula: 32.327.658.
Pedro Diaz, Cédula: 31.887.476.
Sección: S18

Taller #4 Módulo 4 "Realizar un diagrama de la arquitectura en donde se asegure la conexión con la base de datos del proyecto, utilice un diagrama parecido al presentado en la clase"
Proyecto #3 (Monitor de Integridad de Archivos - FIM)

1. Introducción y Objetivo Arquitectónico

El proyecto **FIM-Secure (File Integrity Monitor)** implementa un esquema híbrido de computación segura: combina el rendimiento a bajo nivel del Lenguaje C para el acceso directo a recursos del sistema operativo Windows y cálculo acelerado SHA-256 (vía `Win32 CryptoAPI`), con la flexibilidad de Python para la orquestación, gestión de persistencia en SQLite/JSON y reportería de ciberseguridad. El propósito de este taller es diagramar y documentar la arquitectura que garantiza la conexión segura y confiable entre los módulos de software y la capa de base de datos.

2. Diagrama General de Arquitectura y Flujo de Conectividad



3. Capas y Mecanismos de Seguridad de Conectividad

- **1. Conexión Híbrida C / Python (IPC Seguro)**

  La interacción entre Python y C no utiliza bibliotecas dinámicas propensas a inyección de memoria DLL. Se realiza mediante el módulo estándar subprocess.run() pasando la ruta como un argumento en una lista de parámetros de consola limpios, capturando únicamente STDOUT.

- **2. Conexión Segura Python → Base de Datos**

  La base de datos SQLite/JSON utiliza consultas totalmente parametrizadas mediante el conector nativo de Python (sqlite3). Las entradas nunca se interpolan ni concatenan en código SQL, neutralizando por diseño cualquier ataque de Inyección SQL (SQLi).

- **3. Gestión de Memoria y Manejo de Recursos en C**

  El motor en C implementa verificación de punteros e invalida handles de archivos mediante CloseHandle y liberación explícita del contexto criptográfico (CryptReleaseContext) para evitar fugas de memoria (*memory leaks*).

- **4. Prevención de Buffer Overflow**

  Las operaciones I/O en C se delimitan rigurosamente mediante un buffer estático de lectura de 4096 bytes (BUFSIZE) y buffers de salida fijos de 65 caracteres para almacenar la cadena hexadecimal del SHA-256.

4. Implementación del Conector Parametrizado (Python ➔ SQLite)

El siguiente fragmento muestra la implementación segura de conectividad e inserción transaccional con la base de datos relacional:

```
import sqlite3

class FIMDatabaseConnector:
    def __init__(self, db_path="fim_secure.db"):
        # Establece la conexion con la base de datos SQLite
        self.conn = sqlite3.connect(db_path)
        self.cursor = self.conn.cursor()

    def registrar_alerta_segura(self, id_archivo, tipo_evento, sha256_detectado):
        """Insercion transaccional parametrizada contra SQL Injection"""
```

```
        sql = """
                    INSERT INTO ALERTA (tipo_evento, sha256_detectado, id_archivo,
timestamp)
            VALUES (?, ?, ?, CURRENT_TIMESTAMP);
        """
        try:
            # Los valores se pasan aislados dentro de una tupla parametrizada
            self.cursor.execute(sql, (tipo_evento, sha256_detectado, id_archivo))
            self.conn.commit()
            print(f"[BD OK] Alerta registrada exitosamente para ID Archivo: {id_archivo}")
        except sqlite3.Error as e:
            self.conn.rollback()
            print(f"[BD ERROR] Fallo en transaccion: {e}")

    def cerrar_conexion(self):
        if self.conn:
            self.conn.close()
```

5.Conclusión del Taller N° 4

El diseño de arquitectura presentado demuestra una segregación clara de responsabilidades entre el motor nativo en C, la capa de orquestación en Python y la base de datos. Esta estructura garantiza una conectividad fluida, eficiente y blindada contra vulnerabilidades de memoria y ataques de inyección SQL.